



Mata Kuliah : Pemrograman Framework
Program Studi : D4 – Teknik Informatika
Semester : 6

Kelas : TI-3C
NIM : 2341720165
Nama : Fandy Wahyu Hanzura
Jobsheet Ke- : 15

Laporan Jobsheet

Praktikum 1

```
src > pages > api > auth > TS [...nextauth].ts > authOptions > pages > signIn
1 import NextAuth, { NextAuthOptions } from "next-auth";
2 import CredentialsProvider from "next-auth/providers/credentials";
3
4 export const authOptions: NextAuthOptions = {
5   session: { ...
6 },
7   secret: process.env.NEXTAUTH_SECRET,
8   providers: [ ...
9 ],
10
11   callbacks: { ...
12 },
13
14   pages: {
15     signIn: "/auth/login"
16   }
17 }
18
19 export default NextAuth(authOptions);
20
```

1.



- Menambahkan custom redirect ke halaman login lewat api auth login

```
src > views > auth > login > index.tsx > TampilanRegister
1  import Link from "next/link";
2  import style from "@views/auth/login/login.module.scss";
3  import { useState } from "react";
4  import { useRouter } from "next/router";
5
6  export default function TampilanRegister() {
7    const [isLoading, setIsLoading] = useState(false);
8    const [error, setError] = useState("");
9    const { push } = useRouter();
10
11    const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
12      event.preventDefault();
13      setError("");
14      setIsLoading(true);
15
16      const form = event.currentTarget;
17      const formData = new FormData(form);
18      const data = {
19        email: formData.get("email") as string,
20        fullname: formData.get("Fullname") as string,
21        password: formData.get("Password") as string,
22        role: formData.get("role") as string,
23      };
24
25      const response = await fetch("/api/login", {
```



```
src > views > auth > login > login.module.scss > login
1  .login {
2    min-height: 100vh;
3    display: flex;
4    flex-direction: column;
5    align-items: center;
6    justify-content: center;
7    background: linear-gradient(135deg, #1e3c72, #2a5298);
8    padding: 20px;
9
10   &__title {
11     width: 100%;
12     text-align: center;
13     font-size: 36px;
14     font-weight: 700;
15     color: #ffffff;
16     margin-bottom: 40px;
17     animation: fadeIn 0.8s ease;
18   }
19
20   &__form {
21     background: #ffffff;
22     width: 100%;
23     max-width: 420px;
24     padding: 40px;
25     border-radius: 16px;
```

Halaman login

Email

Fullname

Password

Role

Member

Login

Tidak punya akun? Ke Halaman Register



```
src > views > auth > login > index.tsx > TampilanRegister
6  export default function TampilanRegister() {
44    <div className={style.login}>
45      <h1 className={style.login_title}>Halaman login</h1>
46      <div className={style.login_form}>
47        {error && <p style={{ color: "red", textAlign: "center", marginBottom: "10px" }}>{error}</p>}
48
49        <form onSubmit={handleSubmit}>
50          <div className={style.login_form_item}>
51            <label htmlFor="email" className={style.login_form_item_label}>Email</label>
52            <input type="email" id="email" name="email" placeholder="Email" />
53          </div>
54
55          <div className={style.login_form_item}>
56            <label htmlFor="Password" className={style.login_form_item_label}>Password</label>
57            <input type="password" id="Password" name="Password" placeholder="Password" />
58          </div>
59
60          <button type="submit" disabled={isLoading}>
61            {isLoading ? "Loading..." : "Login"}
62          </button>
63        </form>
64
65        <p className={style.login_link}>
66          Tidak punya akun? <Link href="/auth/register">Ke Halaman Register</Link>
67        </p>
68      </div>
69    </div>
70  }
71
```

Halaman login

Email

Email

Password

Password

Login

Tidak punya akun? Ke Halaman Register



```
src > views > auth > login > index.tsx > TampilanLogin > handleSubmit
7   export default function TampilanLogin() {
15   const handleSubmit = async (event: any) => {
16     event.preventDefault();
17     setError("");
18     setIsLoading(true);
19
20     try {
21       const res = await signIn("credentials", {
22         redirect: false,
23         email: event.target.email.value,
24         password: event.target.password.value,
25         callbackUrl,
26       });
27
28       if (!res?.error) {
29         setIsLoading(false);
30         push(callbackUrl);
31       } else {
32         setIsLoading(false);
33         setError(res.error === "CredentialsSignin" ? "Email or Password wrong" :
34       }
35     } catch (error) {
36       setIsLoading(false);
37       setError("Something went wrong, please try again");
38     }
  }
```

```
src > utils > db > TS servicefirebase.ts > signIn
17
18 > export async function retrieveProducts(collectionName: string) { ...
25 }
26
27 > export async function retrieveProductByID(collectionName: string, id: string) { ...
32 }
33
34
35 > export async function signUp( ...
58 }
59
60 export async function signIn( email: string){
61   const q = query(collection(db, "users"), where("email", "==", email));
62   const querySnapshot = await getDocs(q);
63   const data = querySnapshot.docs.map((doc) => ({ id: doc.id, ...doc.data() }));
64   if (data){
65     return data[0];
66   }else {
67
68     return null;
69   }
70 }
```

- Membuat halaman Login dengan melakukan copy paste melalui template halaman register sebelumnya dan merubah semua nama register menjadi halaman login lalu Mengubah logika pada file index dan melakukan fungsi sign in pada file firebase service



```
10 secret: process.env.NEXTAUTH_SECRET,  
11 providers: [  
12   CredentialsProvider({  
13     name: "Credentials",  
14     credentials: {  
15       // fullname: { label: "Full Name", type: "text" },  
16       email: { label: "Email", type: "email" },  
17       password: { label: "Password", type: "password" }  
18     },  
19  
20     async authorize(credentials) {  
21       if (!credentials?.email || !credentials?.password) return null;  
22  
23       const user: any = await signIn(credentials.email);  
24  
25       if (user) {  
26         const isValid = await bcrypt.compare(  
27           credentials.password,  
28           user.password,  
29         );  
30       }  
31     }  
32   })  
33 ],
```

- 3.
- Mengubah providers untuk bagian login dengan menambahkan bcrypt dan fungsi sign in

```
48 token.fullname = user.fullname;  
49 token.role = user.role;  
50  
51 return token  
52 },  
53  
54 async session({ session, token }: any) {  
55   if (token.email) {  
56     session.user.email = token.email;  
57   }  
58  
59   if (token.fullname) {  
60     session.user.name = token.fullname;  
61   }  
62   if (token.role) {  
63     session.user.role = token.role;  
64   }  
65   return session;  
66 }  
67 },
```

4.



- Mengubah provider dengan memanggil token yang ada sehingga akan bisa login ketika di login menggunakan email dan password



```
src > Middleware > TS withAuth.ts > withAuth > <function>
1  import { getToken } from "next-auth/jwt";
2  import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";
3
4  export default function withAuth(middleware: NextMiddleware, requireAuth: string[] = [])
5  return async (req: NextRequest, next: NextFetchEvent) => {
6      const pathname = req.nextUrl.pathname;
7
8      if (requireAuth.includes(pathname)) {
9          const token = await getToken({
10              req, secret: process.env.NEXTAUTH_SECRET,
11          });
12          if (!token) {
13              const loginUrl = new URL("/auth/login", req.url);
14              loginUrl.searchParams.set("callbackUrl", encodeURIComponent(req.url));
15              return NextResponse.redirect(loginUrl);
16          }
17      }
18      return middleware(req, next);
19  }
20
21 }
```

5.

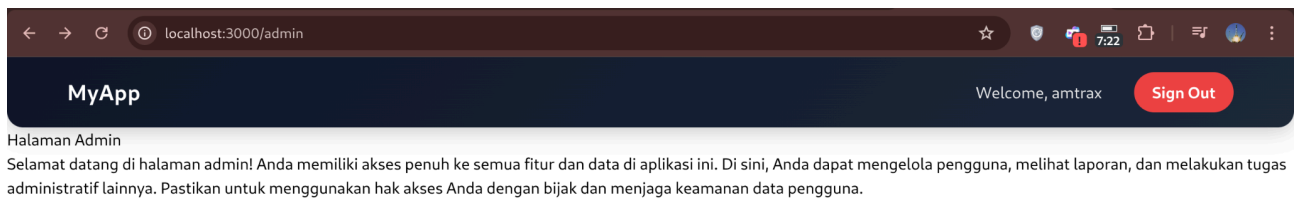
- Menambah parameter loginurl pada file withAuth

```
1  export default function HalamanAdmin() {
2      return (
3          <div>
4              <div className="admin">
5                  <h1>Halaman Admin</h1>
6                  <p>
7                      Selamat datang di halaman admin! Anda memiliki akses penuh ke semua
8                      fitur dan data di aplikasi ini. Di sini, Anda dapat mengelola
9                      pengguna, melihat laporan, dan melakukan tugas administratif lainnya.
10                     Pastikan untuk menggunakan hak akses Anda dengan bijak dan menjaga
11                     keamanan data pengguna.
12                 </p>
13             </div>
14         </div>
15     );
16 }
17
18
```

6.



```
src > Middleware > TS_withAuth.ts > withAuth > <function>
1 import { getToken } from "next-auth/jwt";
2 import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";
3
4 export default function withAuth(middleware: NextMiddleware, requireAuth: string[] = [])
5 return async (req: NextRequest, next: NextFetchEvent) => {
6   const pathname = req.nextUrl.pathname;
7
8   if (requireAuth.includes(pathname)) {
9     const token = await getToken({
10       req, secret: process.env.NEXTAUTH_SECRET,
11     });
12     if (!token) {
13       const loginUrl = new URL("/auth/login", req.url);
14       loginUrl.searchParams.set("callbackUrl", encodeURIComponent(req.url));
15       return NextResponse.redirect(loginUrl);
16     }
17   }
18   if (token.role !== "admin") {
19     const homeUrl = new URL("/", req.url);
20     return NextResponse.redirect(homeUrl);
21   }
22   return middleware(req, next);
23 }
24 }
```



- Menambahkan halaman admin dan check role pada with auth agar hanya admin yang dapat akses

Pertanyaan refleksi

1. Mengapa password harus diverifikasi dengan bcrypt.compare?

Password di database disimpan dalam bentuk **Hash** (acak) demi keamanan. Karena hash bersifat satu arah (*one-way*), kita tidak bisa melihat password aslinya.

- **Mekanisme:** `bcrypt.compare` mengambil password mentah dari input user, melakukan proses hashing ulang dengan *salt* yang tersimpan, lalu membandingkan hasilnya dengan hash di database.
- **Proteksi:** Jika database bocor, penyerang tetap tidak bisa mengetahui password asli pengguna.



2. Mengapa role disimpan di token?

Token (JWT) berfungsi sebagai "kartu identitas" digital yang dibawa browser pada setiap *request*.

- **Efisiensi:** Server tidak perlu melakukan query berulang ke Firestore hanya untuk mengecek hak akses setiap kali user pindah halaman.
- **Stateless:** Memungkinkan aplikasi mengetahui siapa user dan apa perannya secara cepat tanpa membebani performa database.

3. Apa fungsi callbackUrl?

`callbackUrl` berfungsi sebagai parameter navigasi pasca-proses autentikasi.

- **Redirect Otomatis:** Menentukan ke mana user akan diarahkan setelah berhasil **Login** atau **Logout**.
- **UX (User Experience):** Memungkinkan user kembali ke halaman terakhir yang ingin mereka akses sebelum dipaksa masuk ke halaman login.

4. Mengapa middleware penting untuk security?

Middleware adalah "**Gatekeeper**" atau penjaga pintu di level server (sebelum halaman di-render).

- **Pencegahan Dini:** Middleware mengecek validitas token sebelum kode aplikasi dikirim ke browser.
- **Sentralisasi:** Keamanan menjadi terpusat di satu tempat, sehingga tidak perlu menulis logika proteksi manual di setiap file halaman.

5. Apa risiko jika role tidak dicek di middleware?

Tanpa pengecekan di middleware, aplikasi rentan terhadap serangan **Bypass Authentication**.



KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<https://jti.polinema.ac.id>

- **URL Manipulation:** User biasa bisa mengetikkan alamat `/admin` secara manual di browser dan masuk ke dashboard rahasia jika proteksi hanya ada di level UI (menyembunyikan tombol).
- **Client-Side Manipulation:** User dapat memodifikasi state aplikasi di browser (lewat inspect element) untuk mengelabui tampilan, namun mereka tidak bisa mengelabui Middleware yang berjalan di sisi server.